LAB 4 COMPUTER NETWORKS:

ID: 8858 **Name: Halaa Haytham**

ID: 8817 **Name: Nour Magdy**

1. Introduction

User Datagram Protocol (UDP) is a connectionless transport protocol that provides minimal functionality for data transmission. Unlike Transmission Control Protocol (TCP), UDP does not guarantee packet delivery, ordering, or duplication protection. As a result, packets may be lost, corrupted, or received out of order during transmission.

The objective of this project is to design and implement a reliable communication system over UDP by simulating TCP-like behavior in user space. This includes adding mechanisms for:

- Error detection using checksums
- Packet loss detection and retransmission
- Acknowledgment-based reliability
- Basic flow control using a stop-and-wait mechanism
- Support for HTTP protocol communication over UDP

The system also simulates real network conditions such as packet loss and corruption to evaluate reliability mechanisms under unreliable transmission environments.

The design considers trade-offs between performance and reliability, ensuring correct delivery of data while maintaining acceptable efficiency.

---

2. System Design Overview

The system is built as a layered architecture:

- **Application Layer:** HTTP request/response handling (GET and POST)
- **Reliability Layer:** Custom Reliable UDP implementation
- **Transport Layer:** UDP socket communication

2.1 Packet Structure

Each packet contains the following fields:

- seq: Sequence number for ordering
- ack: Acknowledgment number
- flags: Type of packet (DATA, ACK, SYN, SYNACK, FIN)
- data: Payload (HTTP request/response)
- checksum: MD5 hash for error detection

This structure allows simulation of TCP-like reliability over UDP.

---

3. Reliability Mechanisms

3.1 Error Detection (Checksum)

Each packet includes a checksum computed using MD5 hashing of its contents. Upon reception, the checksum is recalculated and compared to detect corruption.

If a mismatch occurs:

- The packet is discarded
- No acknowledgment is sent

This ensures corrupted data is not processed.

3.2 Packet Loss Simulation

Packet loss is simulated probabilistically using a configurable loss probability. If a packet is dropped:

- It is not transmitted
- The sender does not receive an acknowledgment
- A timeout triggers retransmission

3.3 Packet Corruption Simulation

Packet corruption is simulated by modifying packet content or checksum before transmission. Corrupted packets fail checksum validation at the receiver and are discarded.

This tests the robustness of the error detection mechanism.

```
PS C:\Users\me\Desktop\Engineering_Term_8\Networks\lab4_networks> pytho
n test3_packetcorruption.py

[TEST 3] Packet corruption simulation

[CLIENT] SYN sent
[CLIENT] SYN-ACK received
[CLIENT] ACK sent → Connected

[CLIENT] Sending SEQ=0, FLAGS=DATA
[SIMULATION] CORRUPTION GENERATED
[TIMEOUT] Retransmitting...

[CLIENT] Sending SEQ=0, FLAGS=DATA
[SIMULATION] CORRUPTION GENERATED
[TIMEOUT] Retransmitting...

[CLIENT] Sending SEQ=0, FLAGS=DATA
[SIMULATION] CORRUPTION GENERATED
[TIMEOUT] Retransmitting...

[CLIENT] Sending SEQ=0, FLAGS=DATA
[SIMULATION] CORRUPTION GENERATED
[TIMEOUT] Retransmitting...
```

```
TERMINAL
[TIMEOUT] Retransmitting...

[CLIENT] Sending SEQ=0, FLAGS=DATA
[SIMULATION] CORRUPTION GENERATED
[TIMEOUT] Retransmitting...

[CLIENT] Sending SEQ=0, FLAGS=DATA
[SIMULATION] CORRUPTION GENERATED
[TIMEOUT] Retransmitting...

[CLIENT] Sending SEQ=0, FLAGS=DATA
[SIMULATION] CORRUPTION GENERATED
[TIMEOUT] Retransmitting...

[CLIENT] Sending SEQ=0, FLAGS=DATA
[SIMULATION] CORRUPTION GENERATED
[TIMEOUT] Retransmitting...

[CLIENT] Sending SEQ=0, FLAGS=DATA
[SIMULATION] CORRUPTION GENERATED
[TIMEOUT] Retransmitting...

[CLIENT] Sending SEQ=0, FLAGS=DATA
[SIMULATION] CORRUPTION GENERATED
[TIMEOUT] Retransmitting...

[CLIENT] Sending SEQ=0, FLAGS=DATA
[CLIENT DEBUG] got ACK=0, expected=0, valid=True
[SUCCESS] ACK received

[SERVER DEBUG] sending ACK for SEQ=0
[ACK SENT] ACK=0

RESPONSE:
 HTTP/1.0 200 OK

Hello Halaa and Nour! This is a response from the server.

[FIN] Sent
[FIN] ACK received → Closed connection
```

3.4 Acknowledgment and Retransmission

A stop-and-wait protocol is implemented:

1. Sender transmits a packet
2. Sender waits for acknowledgment
3. If ACK is not received within a timeout period:
    - Packet is retransmitted
4. Process repeats until successful delivery

This ensures reliability over unreliable UDP.

3.5 Duplicate Detection

To prevent duplicate processing:

- Each packet includes a sequence number
- Receiver tracks the last received sequence
- Duplicate packets are detected and ignored
- ACK is re-sent for duplicates to prevent sender timeout

3.6 Flow Control

A basic stop-and-wait mechanism is used:

- Only one packet is sent at a time
- Sender waits for ACK before sending next packet

This prevents congestion and simplifies control logic.

---

4. Connection Establishment (Handshake)

A TCP-like three-way handshake is implemented:

1. Client sends SYN
2. Server responds with SYN-ACK
3. Client sends ACK

After this exchange, a reliable connection is established.

---

5. HTTP Layer Implementation

The system supports basic HTTP operations over UDP.

5.1 HTTP GET

Example request:

```
GET / HTTP/1.0
Host: localhost
```

Server response:

```
HTTP/1.0 200 OK
Hello World from Server
```

5.2 HTTP POST

Example request:

```
POST /save HTTP/1.0
Host: localhost
Content-Length: 12

Hello Server
```

The server:

- Parses headers
- Extracts request body
- Returns confirmation response

Example response:

```
HTTP/1.0 200 OK
POST received successfully:
Hello Server
```

---

6. Server Processing Logic

The server continuously:

1. Receives packets
2. Validates checksum
3. Detects duplicates
4. Distinguishes control vs data packets
5. Sends ACK for valid data packets
6. Processes HTTP request

7. Sends HTTP response via reliable UDP layer

---

7. Client Processing Logic

The client:

1. Initiates handshake
2. Sends HTTP request using Reliable UDP
3. Waits for ACK
4. Retransmits if timeout occurs
5. Receives HTTP response
6. Closes connection using FIN handshake

---

8. Test Cases

The system was evaluated using the following scenarios:

8.1 Normal Communication

- No loss or corruption
- Successful delivery on first attempt

8.2 Packet Loss Scenario

- Packets randomly dropped
- Retransmission triggered via timeout
- Successful recovery ensured

8.3 Packet Corruption Scenario

- Packets modified during transmission
- Checksum detection triggered
- Corrupted packets discarded
- Retransmission successful

8.4 HTTP GET Request

- Valid request processed
- Server returns correct response

8.5 HTTP POST Request

- Request body extracted correctly
- Server processes payload
- Response confirms data receipt

---

9. Observations (Wireshark Analysis)

Using Wireshark, UDP packets were captured and analyzed:

- Packets appear as UDP datagrams
- Payload contains JSON-encoded packet structure
- Sequence numbers and flags are visible in payload
- ACK packets confirm successful delivery
- Retransmitted packets are identifiable by repeated sequence numbers

Since UDP does not include built-in reliability, all logic appears in application payload.

No.	Time	Source	Destination	Protocol	Length	Info
69	2095.6344691…	127.0.0.1	127.0.0.1	UDP	128	{"seq": 0, "ack": 0, "flags": "SYN", "data": "", "checksum": "8bfa8fc7a1e08940df82d1cb2c80a92d"}
70	2095.6351162…	127.0.0.1	127.0.0.1	UDP	131	{"seq": 0, "ack": 0, "flags": "SYNACK", "data": "", "checksum": "d8f0b8fff03655db8deb5eb82f8866b7"}
71	2095.6352632…	127.0.0.1	127.0.0.1	UDP	128	{"seq": 0, "ack": 0, "flags": "ACK", "data": "", "checksum": "cbc8e09fa2d5f840d7630a1daa466ade"}
72	2095.6355076…	127.0.0.1	127.0.0.1	UDP	200	{"seq": 0, "ack": 0, "flags": "DATA", "data": "GET / HTTP/1.0\n Host: localhost\n Students: Nour and Halaa\n ", "checksum":
73	2095.6355884…	127.0.0.1	127.0.0.1	UDP	128	{"seq": 0, "ack": 0, "flags": "ACK", "data": "", "checksum": "cbc8e09fa2d5f840d7630a1daa466ade"}
74	2095.6358339…	127.0.0.1	127.0.0.1	UDP	215	{"seq": 0, "ack": 0, "flags": "DATA", "data": "HTTP/1.0 200 OK\n\n Hello Halaa and Nour! This is a response from the server.\n
75	2095.6358822…	127.0.0.1	127.0.0.1	UDP	128	{"seq": 0, "ack": 0, "flags": "ACK", "data": "", "checksum": "cbc8e09fa2d5f840d7630a1daa466ade"}
76	2095.6360668…	127.0.0.1	127.0.0.1	UDP	128	{"seq": 1, "ack": 0, "flags": "FIN", "data": "", "checksum": "e4dce729452a25696e9d743335b77579"}
77	2095.6361607…	127.0.0.1	127.0.0.1	UDP	128	{"seq": 0, "ack": 1, "flags": "ACK", "data": "", "checksum": "635423e389723e5258efe2671c2af6ad"}
78	2098.6390860…	127.0.0.1	127.0.0.1	UDP	128	{"seq": 0, "ack": 0, "flags": "SYN", "data": "", "checksum": "8bfa8fc7a1e08940df82d1cb2c80a92d"}
79	2098.6394869…	127.0.0.1	127.0.0.1	UDP	131	{"seq": 0, "ack": 0, "flags": "SYNACK", "data": "", "checksum": "d8f0b8fff03655db8deb5eb82f8866b7"}
80	2098.6397667…	127.0.0.1	127.0.0.1	UDP	128	{"seq": 0, "ack": 0, "flags": "ACK", "data": "", "checksum": "cbc8e09fa2d5f840d7630a1daa466ade"}
81	2098.6401988…	127.0.0.1	127.0.0.1	UDP	172	{"seq": 0, "ack": 0, "flags": "DATA", "data": "GET / HTTP/1.0\n Host: localhost\n\n ", "checksum": "002693176ca015a89d34cd990f
82	2098.6405098…	127.0.0.1	127.0.0.1	UDP	128	{"seq": 0, "ack": 0, "flags": "ACK", "data": "", "checksum": "cbc8e09fa2d5f840d7630a1daa466ade"}
83	2098.6409825…	127.0.0.1	127.0.0.1	UDP	215	{"seq": 0, "ack": 0, "flags": "DATA", "data": "HTTP/1.0 200 OK\n\n Hello Halaa and Nour! This is a response from the server.\n
84	2098.6410349…	127.0.0.1	127.0.0.1	UDP	128	{"seq": 0, "ack": 0, "flags": "ACK", "data": "", "checksum": "cbc8e09fa2d5f840d7630a1daa466ade"}
85	2098.6412342…	127.0.0.1	127.0.0.1	UDP	128	{"seq": 1, "ack": 0, "flags": "FIN", "data": "", "checksum": "e4dce729452a25696e9d743335b77579"}
86	2098.6413182…	127.0.0.1	127.0.0.1	UDP	128	{"seq": 0, "ack": 1, "flags": "ACK", "data": "", "checksum": "635423e389723e5258efe2671c2af6ad"}
87	2101.6444367…	127.0.0.1	127.0.0.1	UDP	128	{"seq": 0, "ack": 0, "flags": "SYN", "data": "", "checksum": "8bfa8fc7a1e08940df82d1cb2c80a92d"}
88	2101.6452254…	127.0.0.1	127.0.0.1	UDP	131	{"seq": 0, "ack": 0, "flags": "SYNACK", "data": "", "checksum": "d8f0b8fff03655db8deb5eb82f8866b7"}
89	2101.6461230…	127.0.0.1	127.0.0.1	UDP	128	{"seq": 0, "ack": 0, "flags": "ACK", "data": "", "checksum": "cbc8e09fa2d5f840d7630a1daa466ade"}
90	2101.6474513…	127.0.0.1	127.0.0.1	UDP	172	{"seq": 0, "ack": 0, "flags": "DATA", "data": "GET / HTTP/1.0\n Host: localhost\n\n ", "checksum": "002693176ca015a89d34cd990f
91	2101.6476927…	127.0.0.1	127.0.0.1	UDP	128	{"seq": 0, "ack": 0, "flags": "ACK", "data": "", "checksum": "cbc8e09fa2d5f840d7630a1daa466ade"}
92	2101.6488745…	127.0.0.1	127.0.0.1	UDP	215	{"seq": 0, "ack": 0, "flags": "DATA", "data": "HTTP/1.0 200 OK\n\n Hello Halaa and Nour! This is a response from the server.\n
93	2101.6489762…	127.0.0.1	127.0.0.1	UDP	128	{"seq": 0, "ack": 0, "flags": "ACK", "data": "", "checksum": "cbc8e09fa2d5f840d7630a1daa466ade"}
94	2101.6492465…	127.0.0.1	127.0.0.1	UDP	128	{"seq": 1, "ack": 0, "flags": "FIN", "data": "", "checksum": "e4dce729452a25696e9d743335b77579"}
95	2101.6493821…	127.0.0.1	127.0.0.1	UDP	128	{"seq": 0, "ack": 1, "flags": "ACK", "data": "", "checksum": "635423e389723e5258efe2671c2af6ad"}
96	2104.6523963…	127.0.0.1	127.0.0.1	UDP	128	{"seq": 0, "ack": 0, "flags": "SYN", "data": "", "checksum": "8bfa8fc7a1e08940df82d1cb2c80a92d"}
97	2104.6533089…	127.0.0.1	127.0.0.1	UDP	131	{"seq": 0, "ack": 0, "flags": "SYNACK", "data": "", "checksum": "d8f0b8fff03655db8deb5eb82f8866b7"}
98	2104.6540847…	127.0.0.1	127.0.0.1	UDP	128	{"seq": 0, "ack": 0, "flags": "ACK", "data": "", "checksum": "cbc8e09fa2d5f840d7630a1daa466ade"}
99	2104.6553922…	127.0.0.1	127.0.0.1	UDP	219	{"seq": 0, "ack": 0, "flags": "DATA", "data": "POST /save HTTP/1.0\n Host: localhost\n Content-Length: 12\n\n Hello Server\
100	2104.6554845…	127.0.0.1	127.0.0.1	UDP	128	{"seq": 0, "ack": 0, "flags": "ACK", "data": "", "checksum": "cbc8e09fa2d5f840d7630a1daa466ade"}
101	2104.6560409…	127.0.0.1	127.0.0.1	UDP	213	{"seq": 0, "ack": 0, "flags": "DATA", "data": "HTTP/1.0 200 OK\n\n POST received successfully:\n Hello Server\n \n "
102	2104.6561142…	127.0.0.1	127.0.0.1	UDP	128	{"seq": 0, "ack": 0, "flags": "ACK", "data": "", "checksum": "cbc8e09fa2d5f840d7630a1daa466ade"}
103	2104.6562925…	127.0.0.1	127.0.0.1	UDP	128	{"seq": 1, "ack": 0, "flags": "FIN", "data": "", "checksum": "e4dce729452a25696e9d743335b77579"}
104	2104.6563687…	127.0.0.1	127.0.0.1	UDP	128	{"seq": 0, "ack": 1, "flags": "ACK", "data": "", "checksum": "635423e389723e5258efe2671c2af6ad"}

---

10. Results

The system successfully demonstrates:

- Reliable data transmission over UDP
- Recovery from packet loss
- Detection of corrupted packets
- Proper acknowledgment handling
- Functional HTTP communication over UDP

All test cases passed with expected behavior under simulated network conditions.

---

11. Conclusion

This project demonstrates how reliable communication can be built over an unreliable transport protocol such as UDP. By implementing checksums, acknowledgments, retransmissions, and sequence numbering, we successfully simulate TCP-like behavior in user space.

Additionally, by supporting HTTP over UDP, the system shows that application-level protocols can be implemented independently of transport reliability when proper mechanisms are introduced.

The main trade-off observed is between reliability and performance, as retransmissions introduce additional delay but ensure correct delivery.

---

12. Bonus: Extensions and Improvements

Possible future improvements include:

- Sliding window protocol for higher throughput
- Congestion control mechanisms
- Advanced checksum (CRC instead of MD5)
- Multi-client support
- Full HTTP parser compliance